BINARY SEARCH TREE

CODE :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node{
   int data;
   struct node *left;
   struct node *right;
}*bintree ;
//bintree is a pointer to the struct node.

bintree makenode(int val){
   bintree root = (bintree)malloc(sizeof(struct node));
   root->left = NULL;
   root->right = NULL;
   root->data = val;
   return root;
}

bintree insert(bintree t, int data){
   if(!t) //checks if anything is there in location stored in t.
      return makenode(data);
   else if(t->data > data)
      t->left = insert(t->left, data);
   else
      t->right = insert(t->right, data);
   return t;
}
```

NIVEDITHA A| 24BCE2000

```
int search(bintree t, int val){
   if(!t)
      return 0;
   if(t-> data == val)
      return 1;
   if(t->data > val)
      search(t->left, val);
   else
      search(t->right, val);
}

int findmin(bintree t){
   if(!t->left)
      return t->data ;
   findmin(t->left);
}

int findmax(bintree t){
   if(!t->right)
      return t->data ;
   findmax(t->right);
}

bintree delete(bintree t, int val){
   if(t){
      if(t->data == val){
         if(t->right == NULL && t->left == NULL)
            return NULL;
         if(!t->left)
            return t->right ;
```

```
        if(!t->right)
            return t->left;
        t->data = findmin(t->right); //here u are copying the min element in the place of supposed to be deleted element
        t->right = delete(t->right, t->data); //now deleting the min element to avoid repition
        return t;
        }
      else if(t->data > val)
        t->left = delete(t->left, val);
      else
        t->right = delete(t->right, val);
   }
}

void inorder(bintree t){
   if(t != NULL){
      inorder(t->left);
      printf("%d ", t->data);
      inorder(t->right);
   }
}

void preorder(bintree t){
   if(t != NULL){
      printf("%d ", t->data);
      preorder(t->left);
      preorder(t->right);
   }
}
```

```
void postorder(bintree t){
   if(t != NULL){
      postorder(t->left);
      postorder(t->right);
      printf("%d ", t->data);
   }
}

int height(bintree t){
   if(!t)
      return 0;
   int lefth = height(t->left);
   int righth = height(t->right);
   return 1 + (righth>lefth ? righth : lefth);
}

int totalnodes(bintree t){
   if(!t)
      return 0;
   else
      return 1+totalnodes(t->left) + totalnodes(t->right);
}

int leafnodes(bintree t){
   if(!t)
      return 0;
   else if(t->right == NULL && t->right == NULL)
      return 1;
   else
      return leafnodes(t->left) + leafnodes(t->right);
}
```

NIVEDITHA A| 24BCE2000

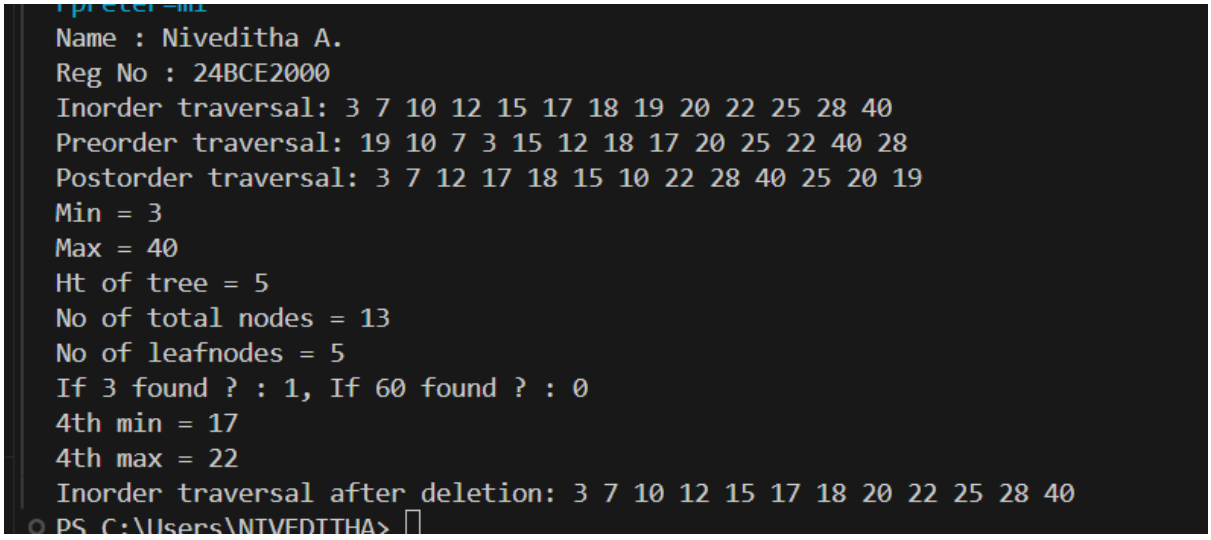
```
void addInArray(bintree t, int arr[], int *i){
   if(t == NULL)
      return;
   addInArray(t->left, arr, i);
   arr[*i] = t->data; //remember to pass address of i, else it will create a new copy of i each time.
   (*i)++;
   addInArray(t->right, arr, i);
}

int kthmax(bintree t, int k){
   int arr[50] = {0};
   int i = 0;
   addInArray(t, arr, &i);
   return arr[i - k];
}

int kthmin(bintree t, int k){
   int arr[50] = {0};
   int i =0;
   addInArray(t, arr, &i);
   return arr[k+1];
}

void main(){
   printf("Name : Niveditha A.\nReg No : 24BCE2000");
   bintree tree = makenode(19);
   tree = insert(tree, 10);
   tree = insert(tree, 7);
   tree = insert(tree, 20);
   tree = insert(tree, 25);
```

```
    tree = insert(tree, 15);
    tree = insert(tree, 18);
    tree = insert(tree, 3);
    tree = insert(tree, 12);
    tree = insert(tree, 40);
    tree = insert(tree, 17);
    tree = insert(tree, 22);
    tree = insert(tree, 28);
    printf("\nInorder traversal: ");
    inorder(tree);
    printf("\nPreorder traversal: ");
    preorder(tree);
    printf("\nPostorder traversal: ");
    postorder(tree);
    printf("\nMin = %d", findmin(tree));
    printf("\nMax = %d", findmax(tree));
    printf("\nHt of tree = %d", height(tree));
    printf("\nNo of total nodes = %d", totalnodes(tree));
    printf("\nNo of leafnodes = %d", leafnodes(tree));
    int res1 = search(tree, 3);
    int res2 = search(tree, 60);
    printf("\nIf 3 found ? : %d, If 60 found ? : %d", res1, res2);
    printf("\n4th min = %d", kthmin(tree, 4));
    printf("\n4th max = %d", kthmax(tree, 4));
    delete(tree, 19);
    printf("\nInorder traversal after deletion: ");
    inorder(tree);
}
```


Name : Niveditha A.
Reg No : 24BCE2000
Inorder traversal: 3 7 10 12 15 17 18 19 20 22 25 28 40
Preorder traversal: 19 10 7 3 15 12 18 17 20 25 22 40 28
Postorder traversal: 3 7 12 17 18 15 10 22 28 40 25 20 19
Min = 3
Max = 40
Ht of tree = 5
No of total nodes = 13
No of leafnodes = 5
If 3 found ? : 1, If 60 found ? : 0
4th min = 17
4th max = 22
Inorder traversal after deletion: 3 7 10 12 15 17 18 20 22 25 28 40